Solution de l'exercice HawQ-ai par SERIGNE SALIOU MBACKE DIOP

1 Méthode 1 : Projected Gradient Descent (PGD) par ensemble de modèles

La première méthode repose sur une attaque adversariale PGD (*Projected Gradient Descent*) appliquée conjointement à un ensemble de dix modèles YOLO via le script `PGD.py`.

Contrairement à une attaque PGD classique appliquée à un seul détecteur, cette approche exploite les gradients de plusieurs modèles afin de rechercher une perturbation commune permettant de réduire les scores associés à la classe *person*.

1.1 Ensemble de modèles

L'ensemble utilisé comprend dix modèles de détection :

$$\mathcal{M} = \left\{ \begin{array}{l} \text{YOLOv5n, YOLOv5x,} \\ \text{YOLOv8n, YOLOv8x,} \\ \text{YOLO11n, YOLO11x,} \\ \text{YOLO12n, YOLO12x,} \\ \text{YOLO26n, YOLO26x} \end{array} \right\}$$

1.2 Fonction de perte de l'ensemble

Soit x l'image originale et δ la perturbation ajoutée à cette image.

L'image adversariale est définie par :

$$x_{\text{adv}} = x + \delta$$

Pour chaque modèle $m \in \mathcal{M}$, les scores associés à la classe *person* sont extraits des prédictions du réseau.

La perte individuelle correspond à la moyenne des scores de cette classe :

$$\mathcal{L}_m(x_{\text{adv}}) = \frac{1}{N} \sum_{i=1}^{N} s^{(m)}_{\text{person},i}(x_{\text{adv}})$$

où N représente le nombre de candidats de détection et $s^{(m)}_{\text{person},i}$ le score associé à la classe *person* pour le candidat i.

La fonction de perte globale est obtenue par addition des pertes individuelles :

$$\mathcal{L}_{\text{ensemble}}(x_{\text{adv}}) = \sum_{m \in \mathcal{M}} \mathcal{L}_m(x_{\text{adv}})$$

L'objectif est de minimiser cette fonction afin de réduire les scores de détection de personnes sur l'ensemble des modèles :

$$\min_{\delta} \mathcal{L}_{\text{ensemble}}(x + \delta)$$

1.3 Mise à jour par gradient

À chaque itération t, le gradient de la fonction de perte globale est calculé par rapport à l'image :

$$g_t = \nabla_{x_t} \mathcal{L}_{\text{ensemble}}(x_t)$$

L'image est ensuite mise à jour par une descente de gradient signée :

$$x_{t+1} = x_t - \alpha \operatorname{sign}(g_t)$$

Une projection garantit que la perturbation reste dans la boule L_∞ autorisée :

$$\delta_{t+1} = \text{clip}\left(x_{t+1} - x, -\varepsilon, \varepsilon\right)$$

L'image est ensuite ramenée dans l'intervalle des valeurs de pixels autorisées :

$$x_{t+1} = \text{clip}(x + \delta_{t+1}, 0, 1)$$

L'algorithme utilise une initialisation aléatoire de la perturbation dans l'intervalle $[-\varepsilon, \varepsilon]$.

1.4 Paramètres expérimentaux

Les paramètres utilisés dans l'implémentation sont :

$$\varepsilon = \frac{8}{255}, \qquad \alpha = \frac{1.5}{255}, \qquad T = 60$$

où ε représente l'amplitude maximale de la perturbation, α le pas de mise à jour et T le nombre d'itérations.

1.5 Évaluation

Après la génération via `PGD.py`, l'image adversariale est évaluée individuellement sur les dix modèles YOLO avec un seuil de confiance de 0.25.

Pour cette première méthode, les images sources `1.avif` et `2.avif` ont été transformées pour donner respectivement les images `pdg_adversarial_1_avif.png` et `pdg_adversarial_2_avif.png`.

1.6 Référence bibliographique

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2018).

Towards Deep Learning Models Resistant to Adversarial Attacks.

International Conference on Learning Representations (ICLR).

`https://arxiv.org/abs/1706.06083`

2 Méthode 2 : Momentum Iterative FGSM (MI-FGSM) par ensemble de modèles

La deuxième méthode repose sur une attaque adversariale MI-FGSM (*Momentum Iterative Fast Gradient Sign Method*) appliquée conjointement à un ensemble de dix modèles YOLO via le script `MI_FGSM.py`.

Contrairement à la première méthode PGD, qui utilise directement le signe du gradient à chaque itération, MI-FGSM intègre un mécanisme de momentum permettant de conserver une partie de la direction des gradients calculés aux itérations précédentes.

2.1 Ensemble de modèles

L'attaque utilise les dix modèles de détection suivants :

$$\mathcal{M} = \left\{ \begin{array}{l} \text{YOLOv5n, YOLOv5x,} \\ \text{YOLOv8n, YOLOv8x,} \\ \text{YOLO11n, YOLO11x,} \\ \text{YOLO12n, YOLO12x,} \\ \text{YOLO26n, YOLO26x} \end{array} \right\}$$

2.2 Fonction de perte de l'ensemble

Soit x l'image originale et x_{adv} l'image adversariale.

Pour chaque modèle $m \in \mathcal{M}$, les scores associés à la classe *person* sont extraits des prédictions du réseau.

La perte individuelle correspond à la moyenne de ces scores :

$$\mathcal{L}_m(x_{\text{adv}}) = \frac{1}{N} \sum_{i=1}^{N} s^{(m)}_{\text{person},i}(x_{\text{adv}})$$

2.3 Mise à jour par momentum

À chaque itération t, le gradient de la fonction de perte globale est calculé par rapport à l'image :

$$g_t^{\text{raw}} = \nabla_{x_t} \mathcal{L}_{\text{ensemble}}(x_t)$$

Le gradient est ensuite normalisé et le mécanisme de momentum est appliqué :

$$G_t = \mu G_{t-1} + \frac{g_t^{\text{raw}}}{\text{mean}(|g_t^{\text{raw}}|)}$$

La mise à jour de l'image est réalisée par une descente de gradient signée :

$$x_{t+1} = x_t - \alpha \, \text{sign}(G_t)$$

2.4 Paramètres expérimentaux

Les paramètres utilisés dans l'implémentation sont :

$$\varepsilon = \frac{8}{255}, \qquad T = 40, \qquad \mu = 1.0, \qquad \alpha = \frac{\varepsilon}{T} \times 1.5$$

2.5 Évaluation

Après la génération via `MI_FGSM.py`, l'image est évaluée sur l'ensemble des dix modèles YOLO avec un seuil de confiance de 0.25.

Pour cette seconde méthode, les images sources `3.png` et `4.jpg` ont été traitées pour générer respectivement les images `fgsm_adversarial_3.png` et `fgsm_adversarial_4_jpg.png`.

2.6 Référence bibliographique

Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X. and Li, J. (2018).

Boosting Adversarial Attacks with Momentum.

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

`https://arxiv.org/abs/1710.06081`

3 Méthode 3 : Carlini & Wagner (C&W) par ensemble de modèles

La troisième méthode repose sur une adaptation de l'approche Carlini & Wagner à un ensemble de dix détecteurs YOLO via le script `CW.py`.

Elle cherche à réduire les scores associés à la classe *person* tout en limitant la distorsion à l'aide d'un changement de variable tanh.

3.1 Ensemble de modèles

L'ensemble utilisé comprend les dix modèles suivants :

$$\mathcal{M} = \left\{ \begin{array}{ll} \text{YOLOv5n}, & \text{YOLOv5x}, \\ \text{YOLOv8n}, & \text{YOLOv8x}, \\ \text{YOLO11n}, & \text{YOLO11x}, \\ \text{YOLO12n}, & \text{YOLO12x}, \\ \text{YOLO26n}, & \text{YOLO26x} \end{array} \right\}$$

3.2 Fonction objectif

La fonction objectif combine la distorsion et la pénalisation des scores de détection :

$$\mathcal{L}(x, \delta) = \frac{1}{N}\|\delta\|_2^2 + c\,\mathcal{L}_{\text{ensemble}}(x + \delta)$$

avec un changement de variable pour respecter la norme L_∞ :

$$\delta = \varepsilon \tanh(w)$$

3.3 Paramètres expérimentaux

Les paramètres utilisés dans l'implémentation sont :

$$\varepsilon = \frac{32}{255}, \qquad \alpha = 2 \times 10^{-2}, \qquad T = 200, \qquad c = 100$$

3.4 Évaluation

Après l'optimisation via `CW.py`, l'image finale est évaluée sur les dix modèles YOLO avec un seuil de confiance de 0.25.

Pour cette troisième méthode, l'image source `4.avif` a été modifiée pour produire l'image `CW_adversarial_4_avif.png`.

3.5 Référence bibliographique

Carlini, N. and Wagner, D. (2017).

Towards Evaluating the Robustness of Neural Networks.

IEEE Symposium on Security and Privacy.

`https://arxiv.org/abs/1608.04644`

Remarque importante sur l'intégrité des données

Toutes les images transformées issues des différents scripts d'attaques (`PGD.py`, `MI_FGSM.py` et `CW.py`) sont systématiquement enregistrées au format **PNG sans perte**.

Cette précaution est indispensable pour empêcher qu'une compression avec perte (telle que celle du format JPEG et celle du avif) n'altère, ne lisse ou ne détruise les modifications extrêmement fines du bruit contradictoire L_∞, garantissant ainsi la persistance et l'efficacité réelle des perturbations lors des évaluations ultérieures.